CASE STUDY

Question 1

A logistics company is developing an RL-based warehouse robot that learns to pick, pack, and transport items efficiently while avoiding obstacles and minimizing delays. Analyze how the components of a Markov Decision Process (states, actions, rewards, policy, and environment) can be defined for this system. Design a reward function that balances speed, accuracy, and safety, and evaluate how reward design influences the robot's learning and behavior.

Answer

Understanding of the Case

The warehouse robot must autonomously complete tasks such as picking, packing, and transporting items. Reinforcement Learning enables the robot to learn an optimal strategy through interaction with the warehouse environment.

MDP Components

Component	Description
States (S)	Robot position, battery level, obstacle locations, package location, destination, task completion status
Actions (A)	Move forward, backward, left, right, pick item, place item, recharge battery, avoid obstacle
Environment	Warehouse containing shelves, workers, charging stations, and obstacles
Policy (π)	Chooses the best action for each state to maximize long-term reward
Reward (R)	Positive rewards for successful delivery and negative rewards for collisions or delays

Reward Function

- Successfully pick item = **+20**
- Correct delivery = **+50**
- Shortest path followed = **+10**
- Collision with obstacle = **−40**
- Wrong item picked = **−30**
- Time delay = **−2 per step**
- Battery exhausted = **−50**

Analysis

The reward function encourages the robot to finish tasks quickly while maintaining safety and accuracy. High rewards motivate efficient deliveries, whereas penalties discourage unsafe behavior and unnecessary movements. Over many training episodes, the robot learns the optimal navigation strategy.

Solution Design

A **Q-Learning** algorithm is suitable because the warehouse environment can be modeled using discrete states and actions. The Q-table is updated continuously until the robot learns the best policy for efficient task execution.

Evaluation

A balanced reward function improves learning speed, reduces collisions, minimizes delivery time, and increases operational efficiency. Improper reward design may lead to unsafe shortcuts or inefficient behavior.

Question 2

A streaming platform wants to use Reinforcement Learning to recommend movies dynamically based on user preferences and viewing history. Apply RL concepts to construct a suitable state representation, action space, and reward function. Analyze how different reward signals affect user engagement and evaluate the trade-off between exploration and exploitation.

Understanding of the Case

The objective is to recommend personalized movies that maximize user satisfaction and viewing time. Reinforcement Learning adapts recommendations according to user feedback.

RL Framework

Component	Description
States	User viewing history, favorite genres, watch duration, ratings, search history, time of day
Actions	Recommend a movie, recommend a genre, recommend a series, recommend trending content
Environment	Users interacting with the streaming platform
Policy	Select recommendations that maximize long-term engagement
Reward	Based on user interactions and satisfaction

Reward Function

- Movie watched completely = **+15**
- Movie liked = **+10**
- Positive rating = **+8**
- User subscribes = **+20**
- Recommendation ignored = **−5**
- Recommendation skipped = **−8**

Analysis

Positive rewards increase when recommendations match user interests. Negative rewards discourage poor recommendations. Continuous learning improves personalization and customer retention.

Exploration vs Exploitation

Exploration

- Suggest new genres
- Discover changing user interests
- Improves recommendation diversity

Exploitation

- Recommend favorite genres
- Higher immediate reward
- Increases short-term user satisfaction

A balanced ε-greedy strategy combines both approaches for better long-term performance.

Solution Design

A **Deep Q-Network (DQN)** is appropriate because user preferences involve large and complex state spaces. Neural networks estimate Q-values efficiently.

Evaluation

Effective reward signals improve user engagement, viewing duration, and subscription retention. Excessive exploitation may reduce content diversity, whereas too much exploration may decrease user satisfaction.

Question 3

An autonomous agricultural system uses RL to optimize irrigation and fertilizer usage based on soil conditions and weather forecasts. Design an RL framework by defining states, actions,

and rewards. Analyze the effect of delayed rewards on crop yield optimization and justify the choice of a suitable learning algorithm.

Understanding of the Case

The system aims to improve agricultural productivity while minimizing water and fertilizer consumption through intelligent decision-making.

RL Framework

Component	Description
States	Soil moisture, temperature, humidity, rainfall prediction, crop growth stage, nutrient level
Actions	Increase irrigation, reduce irrigation, apply fertilizer, postpone irrigation, no action
Environment	Agricultural field with changing weather and soil conditions
Policy	Select actions that maximize crop yield and resource efficiency
Reward	Crop growth and efficient resource utilization

Reward Function

- Healthy crop growth = **+40**
- High crop yield = **+60**
- Efficient water usage = **+20**
- Excess irrigation = **−20**
- Fertilizer wastage = **−25**
- Crop damage = **−50**

Analysis

Agricultural rewards are delayed because crop yield becomes visible only after several weeks or months. The RL agent must learn which early decisions contribute to future rewards despite delayed feedback.

Solution Design

A **Deep Q-Network (DQN)** is suitable because agricultural environments contain many continuous variables. DQN handles large state spaces and learns complex relationships effectively.

Evaluation

Delayed rewards increase learning complexity, but RL can still identify optimal irrigation and fertilizer strategies that improve yield while conserving resources.

Question 4

A cybersecurity system uses RL to detect and respond to network attacks in real time. Analyze how states and environment can be modeled for intrusion detection. Design appropriate actions and reward functions, and evaluate the challenges of sparse rewards and real-time decision-making.

Understanding of the Case

The cybersecurity system continuously monitors network traffic and learns how to detect malicious activities while minimizing false alarms.

RL Framework

Component	Description
States	Network traffic, packet rate, login attempts, firewall logs, suspicious IP addresses
Actions	Block IP, allow traffic, alert administrator, isolate system, continue monitoring
Environment	Enterprise computer network
Policy	Choose actions that maximize network security
Reward	Rewards based on successful attack prevention

Reward Function

- Correctly detect attack = **+50**
- Successfully block attacker = **+40**
- Prevent data breach = **+60**
- False alarm = **−20**
- Missed attack = **−80**
- Unnecessary blocking = **−15**

Analysis

Cyber attacks occur infrequently, leading to sparse rewards. The agent may receive long periods without positive feedback, making learning slower. Real-time decisions also require rapid action to minimize damage.

Solution Design

A **Deep Q-Network (DQN)** is suitable because cybersecurity involves large state spaces and dynamic attack patterns. Experience replay improves learning efficiency.

Evaluation

Proper reward design helps reduce false positives and improves attack detection accuracy. Real-time RL enhances network security but requires fast computation and continuous adaptation.

Question 5

A ride-sharing company plans to use RL to optimize driver allocation and dynamic pricing strategies. Apply RL concepts to define states, actions, and rewards. Analyze how environmental uncertainty affects learning and evaluate ethical concerns related to pricing strategies.

Understanding of the Case

The objective is to efficiently allocate drivers and determine fair ride prices while adapting to changing demand and traffic conditions.

RL Framework

Component	Description
States	Driver locations, passenger requests, traffic conditions, weather, time of day, demand level
Actions	Assign driver, adjust fare, relocate drivers, accept ride request, reject request
Environment	City transportation network
Policy	Maximize revenue while minimizing passenger waiting time
Reward	Efficient service and customer satisfaction

Reward Function

- Successful ride completion = **+40**
- Reduced passenger waiting time = **+20**
- Efficient driver utilization = **+15**
- Ride cancellation = **−30**
- Long waiting time = **−20**
- Unfair surge pricing = **−25**

Analysis

Traffic congestion, weather, and fluctuating passenger demand create uncertainty in the environment. The RL agent continuously updates its policy to adapt to these changing conditions.

Solution Design

A **Deep Reinforcement Learning (DQN)** approach is appropriate because the transportation environment is dynamic and contains a large number of states. The model learns optimal pricing and driver allocation strategies over time.

Evaluation

RL improves ride allocation efficiency, reduces passenger waiting time, and increases driver utilization. Ethical concerns include excessive surge pricing, customer fairness, and transparency. Reward functions should balance company profit with customer satisfaction and fairness.